

Python Basics → OOPs: Practical Refresher

A practical, code-first revision guide for rebuilding Python fundamentals and mastering Object-Oriented Programming

Focus: understanding → writing code → recognizing patterns in real projects

How to Use This Guide

This is designed as a refresher rather than a purely theoretical textbook. For each topic, first understand the explanation, then type the example yourself, change it, break it, and fix it. The goal is to make the concepts usable when you read or write real Python code.

- Do not only read the code. Type it manually at least once.
- After every example, change one requirement and see what breaks.
- When you see a class in a project, identify its attributes, methods, relationships, and responsibilities.
- Prefer simple Python first. Add advanced patterns only when they solve a real problem.
- Use the final practical project to connect all OOP concepts together.

Part 1 — Python Foundations

1. What Is Python?

Python is a high-level, general-purpose programming language created with an emphasis on readable syntax and developer productivity.

It is dynamically typed, interpreted/bytecode-compiled, garbage-collected, and supports multiple programming styles including procedural, functional, and object-oriented programming.

A Python program is usually written in .py files and executed by the Python interpreter. Python code is commonly used for web development, automation, data engineering, AI/ML, scripting, testing, APIs, and many other areas.

Key points: Readable syntax • Dynamically typed • Multi-paradigm • Large ecosystem • Automatic memory management

Detailed practical example:

```
name = "Srinadh"
age = 24

print(f"Hello, {name}!")
print("Next year:", age + 1)
```

Why this matters in real projects: In an AI/ML project, Python often connects data processing, model code, APIs, background jobs, and business logic in the same application.

2. Python History — Why Python Became Popular

Python was created by Guido van Rossum and first released publicly in 1991.

Its design emphasized simplicity, readability, and rapid development. Python 2 and Python 3 coexisted for years, but Python 2 reached end-of-life in 2020. Modern development should use Python 3.

Python became especially influential because its simple language was combined with a huge ecosystem. Libraries such as NumPy, pandas, Django, Flask, FastAPI, PyTorch, and scikit-learn made Python useful far beyond scripting.

Key points: Guido van Rossum • First public release: 1991 • Python 3 is the modern standard • Python 2 ended in 2020

Detailed practical example:

```
# Modern Python 3 example
def calculate_average(values):
    return sum(values) / len(values)

scores = [80, 90, 75, 95]
average = calculate_average(scores)

print(f"Average score: {average:.2f}")
```

Why this matters in real projects: The important takeaway is not memorizing historical dates. It is understanding why Python is still widely used: readable code, fast development, and an enormous library ecosystem.

3. Variables and Dynamic Typing

A variable in Python is a name referring to an object. Python does not require you to declare a variable's type before assigning a value. The type belongs to the object, not the variable name.

This makes Python flexible, but you must understand what type of object a variable currently refers to.

Detailed practical example:

```
value = 10
print(type(value))

value = "hello"
print(type(value))

value = [1, 2, 3]
print(type(value))

# The same variable name can refer to different object types.
```

Why this matters in real projects: Dynamic typing speeds up development, but type mistakes can appear at runtime. Type hints can make large projects easier to maintain.

4. Core Data Types

The most important built-in types for everyday Python are int, float, bool, str, list, tuple, set, dict, and None.

Lists are mutable sequences. Tuples are immutable sequences. Sets store unique values. Dictionaries store key-value mappings. Strings are immutable sequences of characters.

Detailed practical example:

```
age = 24                # int
price = 99.50           # float
is_active = True        # bool
name = "Srinadh"        # str
skills = ["Python", "SQL"] # list
point = (10, 20)        # tuple
tags = {"ai", "python"}  # set
user = {"name": "Srinadh", "age": 24} # dict
result = None

print(user["name"])
skills.append("FastAPI")
print(skills)
```

Why this matters in real projects: These types appear everywhere in APIs, database code, configuration, ML pipelines, and application logic.

5. Conditions and Loops

Conditions allow a program to make decisions. Loops allow repeated work.

The most common constructs are if/elif/else, for, and while. Python's for loop is especially common because it iterates directly over objects rather than requiring index management in most cases.

Detailed practical example:

```
users = [
    {"name": "A", "active": True},
    {"name": "B", "active": False},
    {"name": "C", "active": True},
]

for user in users:
    if user["active"]:
        print(user["name"], "is active")
    else:
        print(user["name"], "is inactive")
```

Why this matters in real projects: Most business logic eventually contains conditions and loops. OOP does not replace these basics; OOP organizes them inside reusable objects.

6. Functions

A function groups reusable behavior. Functions receive inputs through parameters and can return a result. A good function generally has one clear responsibility and a meaningful name.

Detailed practical example:

```
def calculate_total(price, quantity, tax_rate=0.18):
    subtotal = price * quantity
    tax = subtotal * tax_rate
    return subtotal + tax

total = calculate_total(500, 2)
print(total)
```

Why this matters in real projects: Classes are built from functions called methods. If your function fundamentals are weak, OOP will feel confusing because methods are simply functions attached to objects/classes.

7. Exceptions

Exceptions represent errors or unusual conditions that interrupt normal execution.

Use try/except when you can meaningfully handle an error. Avoid catching every exception blindly because that can hide real bugs.

Detailed practical example:

```
def divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return None

result = divide(10, 0)

if result is None:
    print("Cannot divide by zero")
else:
    print(result)
```

Why this matters in real projects: Real APIs, database operations, file handling, and external services fail sometimes. Good Python code anticipates expected failures and handles them explicitly.

Part 2 — The Mental Model of OOP

8. What Is OOP?

Object-Oriented Programming (OOP) is a way of organizing software around objects that combine data and behavior. Instead of keeping all data and functions separate, an object can own its state and expose operations that make sense for that state.

Think about a real application:

- User → has name, email, status; can log in, update profile, deactivate.
- BankAccount → has balance; can deposit and withdraw.
- Order → has items and status; can calculate total and change status.
- MLModel → has model configuration; can train, predict, save, and load.

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def show_balance(self):
        print(f"{self.owner}: ₹{self.balance}")

account = BankAccount("Srinadh", 1000)
account.deposit(500)
account.show_balance()
```

The key idea: the account object stores its own data and provides operations that work on that data.

Python Basics → OOPs Practical Refresher

9. Class

A class is a blueprint or definition used to create objects. It describes what data an object can hold and what operations it can perform.

A class itself is not usually the individual entity you work with. Instances created from the class are the actual objects.

Detailed practical example:

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def is_passed(self):
        return self.marks >= 40

student = Student("Ravi", 72)

print(student.name)
print(student.is_passed())
```

Why this matters in real projects: When you repeatedly need the same structure and behavior for many entities, a class is often a good fit.

10. Object / Instance

An object is an instance of a class. You can create many independent objects from one class. Each object can have different state while sharing the same class behavior.

Detailed practical example:

```
class Employee:
    def __init__(self, name, role):
        self.name = name
        self.role = role

employee1 = Employee("Ravi", "Backend Developer")
employee2 = Employee("Anu", "ML Engineer")

print(employee1.name, employee1.role)
print(employee2.name, employee2.role)
```

Why this matters in real projects: In a web application, one User class may produce thousands or millions of user objects conceptually, even if database/ORM implementation details vary.

11. `__init__()` Constructor / Initializer

`__init__` is the initializer method automatically called after an object is created. It is commonly used to establish the object's initial state.

Technically, object creation involves `__new__` and initialization involves `__init__`. For normal Python application development, `__init__` is the method you will use most often.

Detailed practical example:

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

product = Product("Keyboard", 1500)

print(product.name)
print(product.price)
```

Why this matters in real projects: A well-designed `__init__` makes the object's required state explicit. Avoid stuffing unrelated business logic into it.

12. self — The Current Object

`self` refers to the current instance. It lets methods access data and other methods belonging to that object.

Python does not automatically hide `self` from the method definition. You explicitly write it as the first parameter of normal instance methods.

Detailed practical example:

```
class User:
    def __init__(self, name):
        self.name = name

    def greet(self):
        return f"Hello, {self.name}"

user = User("Srinadh")
print(user.greet())

# Conceptually, Python turns:
# user.greet()
# into something similar to:
# User.greet(user)
```

Why this matters in real projects: The most common beginner mistake is forgetting `self` when defining instance methods.

13. Instance Attributes and Instance Methods

Instance attributes belong to individual objects. Instance methods operate on a particular object and normally receive `self`.

Changing one object's instance attribute does not automatically change another object's attribute.

Detailed practical example:

```
class Car:
    def __init__(self, brand, speed=0):
        self.brand = brand
        self.speed = speed

    def accelerate(self, amount):
        self.speed += amount

car1 = Car("Toyota")
car2 = Car("Honda")

car1.accelerate(30)

print(car1.speed) # 30
print(car2.speed) # 0
```

Why this matters in real projects: Most business objects are primarily built from instance state plus instance methods.

14. Class Attributes

A class attribute is stored on the class and can be shared by instances. It is useful for values common to all instances, such as a default configuration or a constant.

Detailed practical example:

```
class Employee:
    company = "Example Corp"

    def __init__(self, name):
        self.name = name

e1 = Employee("Ravi")
e2 = Employee("Anu")

print(e1.company)
print(e2.company)

Employee.company = "New Corp"

print(e1.company)
print(e2.company)
```

Why this matters in real projects: Be careful with mutable class attributes such as lists or dictionaries: all instances can share the same object, which can cause unexpected state.

15. Instance Method

An instance method receives self and works with the state of a particular object. This is the default method type you will write in most classes.

Detailed practical example:

```
class BankAccount:
    def __init__(self, balance):
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

account = BankAccount(1000)
account.deposit(500)

print(account.balance)
```

Why this matters in real projects: Use an instance method when the operation needs the object's own state.

16. Class Method — @classmethod

A class method receives cls instead of self. It operates on the class itself rather than one specific instance. It is commonly used for alternative constructors and class-level behavior.

Detailed practical example:

```
class User:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    @classmethod
    def from_string(cls, data):
        name, age = data.split(",")
        return cls(name, int(age))

user = User.from_string("Srinadh,24")

print(user.name)
print(user.age)
```

Why this matters in real projects: A class method is especially useful when you want multiple clean ways to construct an object from different input formats.

17. Static Method — @staticmethod

A static method does not automatically receive self or cls. It is placed inside a class because it is logically related to that class, but it does not need object or class state.

Detailed practical example:

```
class UserValidator:
    @staticmethod
    def is_valid_email(email):
        return "@" in email and "." in email.split("@")[-1]

print(UserValidator.is_valid_email("test@example.com"))
print(UserValidator.is_valid_email("invalid"))
```

Why this matters in real projects: Use static methods for small utilities that conceptually belong to a class but do not need instance/class state.

Part 3 — The Four Core OOP Pillars

18. Encapsulation

Encapsulation means keeping data and the operations that manage that data together, while controlling how internal state is accessed or modified.

Python does not enforce private fields in the same way as some languages. A leading underscore communicates 'internal use', while double underscore triggers name mangling. In practice, properties and methods are often used to control access.

Key points: Keep related state + behavior together • Expose a clear public interface • Protect important invariants

Detailed practical example:

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self._balance = balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Amount must be positive")
        self._balance += amount

    def get_balance(self):
        return self._balance

account = BankAccount("Srinadh", 1000)
account.deposit(500)

print(account.get_balance())
```

Why this matters in real projects: Encapsulation prevents the rest of the application from changing important state arbitrarily. Instead of directly modifying balance, callers use deposit(), where validation can be enforced.

19. Properties — @property

A property lets you expose method-based logic using attribute-like syntax. It is one of the most practical Python tools for encapsulation and validation.

Detailed practical example:

```
class Employee:
    def __init__(self, salary):
        self._salary = salary

    @property
    def salary(self):
        return self._salary

    @salary.setter
    def salary(self, value):
        if value < 0:
            raise ValueError("Salary cannot be negative")
        self._salary = value

employee = Employee(50000)
print(employee.salary)

employee.salary = 60000
print(employee.salary)
```

Why this matters in real projects: Properties are useful when you want simple attribute syntax now but need validation, computed values, logging, or other logic behind the scenes later.

20. Abstraction

Abstraction means exposing what an object can do while hiding unnecessary implementation details.

For example, a payment service can expose `pay(amount)` without forcing callers to understand HTTP requests, authentication, retries, or gateway-specific details.

Detailed practical example:

```
from abc import ABC, abstractmethod

class PaymentGateway(ABC):
    @abstractmethod
    def pay(self, amount):
        pass

class RazorpayGateway(PaymentGateway):
    def pay(self, amount):
        print(f"Processing ₹{amount} using Razorpay")

class StripeGateway(PaymentGateway):
    def pay(self, amount):
        print(f"Processing ₹{amount} using Stripe")

def checkout(gateway, amount):
    gateway.pay(amount)

checkout(RazorpayGateway(), 1000)
checkout(StripeGateway(), 2000)
```

Why this matters in real projects: Abstraction is valuable when your application needs to work with multiple implementations behind the same interface.

21. Inheritance

Inheritance allows a child class to reuse and extend behavior from a parent class. The child class can inherit methods and attributes, add new behavior, or override inherited behavior.

Use inheritance when there is a genuine 'is-a' relationship and shared behavior makes the design clearer.

Detailed practical example:

```
class Employee:
    def __init__(self, name):
        self.name = name

    def work(self):
        print(self.name, "is working")

class Developer(Employee):
    def write_code(self):
        print(self.name, "is writing Python code")

developer = Developer("Srinadh")

developer.work()
developer.write_code()
```

Why this matters in real projects: Inheritance can reduce duplication, but deep inheritance trees often become difficult to maintain. Prefer composition when it represents the domain more naturally.

22. Types of Inheritance

Common inheritance structures include single inheritance, multilevel inheritance, multiple inheritance, and hierarchical inheritance.

Python supports all of them, but practical code should favor simple, understandable relationships.

Detailed practical example:

```
# Single inheritance
class Animal:
    def eat(self):
        print("Eating")

class Dog(Animal):
    def bark(self):
        print("Barking")

# Multilevel inheritance
class Puppy(Dog):
    def play(self):
        print("Playing")

# Hierarchical inheritance
class Cat(Animal):
    def meow(self):
        print("Meow")

dog = Dog()
puppy = Puppy()
cat = Cat()

dog.eat()
puppy.eat()
cat.eat()
```

Why this matters in real projects: Knowing the types helps you understand existing code. It does not mean you should deliberately create complicated inheritance structures.

23. super()

super() gives a child class access to behavior from its parent according to Python's method resolution order. It is commonly used to extend a parent initializer or method rather than replacing it completely.

Detailed practical example:

```
class Employee:
    def __init__(self, name):
        self.name = name

    def describe(self):
        print(f"Employee: {self.name}")

class Developer(Employee):
    def __init__(self, name, language):
        super().__init__(name)
        self.language = language

    def describe(self):
        super().describe()
        print(f"Language: {self.language}")

developer = Developer("Srinadh", "Python")
developer.describe()
```

Why this matters in real projects: `super()` helps you reuse parent behavior while adding child-specific behavior.

24. Polymorphism

Polymorphism means the same operation/interface can work with different object types. Python often achieves this naturally through duck typing: if an object supports the required behavior, the function can use it.

Key points: Same interface • Different implementations • Duck typing • Flexible code

Detailed practical example:

```
class Dog:
    def speak(self):
        return "Woof"

class Cat:
    def speak(self):
        return "Meow"

class Robot:
    def speak(self):
        return "Beep"

def make_speak(entity):
    print(entity.speak())

make_speak(Dog())
make_speak(Cat())
make_speak(Robot())
```

Why this matters in real projects: This is extremely common in Python. Functions often care about what an object can do rather than what exact class it belongs to.

25. Method Overriding

Method overriding occurs when a child class provides its own implementation of a method defined by its parent class.

Detailed practical example:

```
class Notification:
    def send(self):
        print("Sending notification")

class EmailNotification(Notification):
    def send(self):
        print("Sending email")

class SMSNotification(Notification):
    def send(self):
        print("Sending SMS")

notifications = [
    EmailNotification(),
    SMSNotification(),
]

for notification in notifications:
    notification.send()
```

Why this matters in real projects: Overriding lets each specialized class provide behavior appropriate to its implementation while callers use a common interface.

26. Method Overloading in Python

Traditional languages may support multiple methods with the same name but different parameter signatures. Python does not support that style of method overloading directly.

Instead, Python commonly uses default arguments, *args, **kwargs, or singledispatch when different inputs need different behavior.

Detailed practical example:

```
class Calculator:
    def add(self, a, b=0, c=0):
        return a + b + c

calculator = Calculator()

print(calculator.add(10))
print(calculator.add(10, 20))
print(calculator.add(10, 20, 30))

# Another flexible approach:
def multiply(*numbers):
    result = 1
    for number in numbers:
        result *= number
    return result

print(multiply(2, 3))
print(multiply(2, 3, 4))
```

Why this matters in real projects: Do not force Java-style overload patterns into Python. Use Python's flexible argument system when it makes the API clearer.

Part 4 — OOP Relationships and Design

27. Composition — 'Has-a'

Composition means an object contains or uses another object as one of its parts. It is often more flexible than inheritance.

Example: a Car has an Engine. A car is not an engine, so inheritance would model the relationship incorrectly.

Detailed practical example:

```
class Engine:
    def start(self):
        print("Engine started")

class Car:
    def __init__(self):
        self.engine = Engine()

    def start(self):
        self.engine.start()
        print("Car started")

car = Car()
car.start()
```


Why this matters in real projects: Composition is one of the most important practical OOP ideas. In production systems, complex services are often assembled from smaller components rather than huge inheritance trees.

28. Aggregation

Aggregation is a weaker 'has-a' relationship where the contained object can exist independently of the container.

For example, a Department can have Employees, but an Employee can exist even if the Department object is removed.

Detailed practical example:

```
class Employee:
    def __init__(self, name):
        self.name = name

class Department:
    def __init__(self, name, employees):
        self.name = name
        self.employees = employees

employees = [
    Employee("Ravi"),
    Employee("Anu"),
]

backend = Department("Backend", employees)

for employee in backend.employees:
    print(employee.name)
```

Why this matters in real projects: The distinction between composition and aggregation is mostly about ownership/lifecycle. In everyday Python code, both are usually represented by object references.

29. Association

Association simply means two objects know about or interact with each other. There does not have to be strong ownership.

For example, a Doctor and Patient can interact without either object containing the other as an essential part.

Detailed practical example:

```
class Doctor:
    def __init__(self, name):
        self.name = name

    def consult(self, patient):
        print(f"{self.name} is consulting {patient.name}")

class Patient:
    def __init__(self, name):
        self.name = name

doctor = Doctor("Dr. Rao")
patient = Patient("Kiran")

doctor.consult(patient)
```

Why this matters in real projects: When designing a system, ask whether one object owns another, contains it, or simply collaborates with it.

Part 5 — Python OOP Features You Will See in Real Code

30. Magic / Dunder Methods

Special methods have names surrounded by double underscores, such as `__init__`, `__str__`, `__len__`, and `__eq__`. Python calls these methods in response to language operations.

You do not usually call them directly. You implement them to make your custom objects behave naturally with Python.

Detailed practical example:

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    def __str__(self):
        return f"{self.name} - ₹{self.price}"

    def __eq__(self, other):
        if not isinstance(other, Product):
            return NotImplemented
        return self.name == other.name and self.price == other.price

p1 = Product("Keyboard", 1500)
p2 = Product("Keyboard", 1500)

print(p1)
print(p1 == p2)
```

Why this matters in real projects: When you print an object, compare it, check its length, iterate over it, or use operators, Python may invoke special methods behind the scenes.

31. `__str__` vs `__repr__`

`__str__` is intended for a readable representation, usually for users. `__repr__` is intended to be a more developer-oriented representation useful for debugging.

If `__str__` is not defined, Python can fall back to `__repr__` for some display contexts.

Detailed practical example:

```
class User:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"User: {self.name}"

    def __repr__(self):
        return f"User(name={self.name!r}, age={self.age!r})"

user = User("Srinadh", 24)

print(str(user))
print(repr(user))
```

Why this matters in real projects: Good representations make debugging much easier, especially when objects appear inside lists, logs, or exception messages.

32. `__len__` and `__iter__`

Special methods can make your custom classes work naturally with Python's built-in operations. `__len__` controls `len(obj)`, while `__iter__` lets an object participate in for loops.

Detailed practical example:

```
class Team:
    def __init__(self, members):
        self.members = members

    def __len__(self):
        return len(self.members)

    def __iter__(self):
        return iter(self.members)

team = Team(["Ravi", "Anu", "Kiran"])

print(len(team))

for member in team:
    print(member)
```

Why this matters in real projects: This is useful when building domain objects that should feel like normal Python collections.

33. Private-ish Attributes: `_name` and `__name`

A single leading underscore is a convention meaning 'internal use'. Python does not prevent access.

A double leading underscore triggers name mangling. It is mainly used to avoid accidental name collisions in subclasses, not to provide strong security.

Detailed practical example:

```
class Account:
    def __init__(self):
        self._balance = 1000
        self.__pin = "1234"

    def check_pin(self, pin):
        return pin == self.__pin

account = Account()

print(account._balance)          # Convention says: internal
print(account.check_pin("1234"))

# account.__pin would not access the attribute normally.
# Python internally mangles the name to something like _Account__pin.
```

Why this matters in real projects: Do not treat underscores as a security mechanism. They communicate design intent and provide name-mangling behavior.

34. Dataclasses

dataclasses reduce boilerplate for classes that primarily store data. Python can automatically generate `__init__`, `__repr__`, equality methods, and more.

They are not a replacement for normal classes; they are especially useful for data-oriented objects.

Detailed practical example:

```
from dataclasses import dataclass

@dataclass
class User:
    name: str
    age: int
    active: bool = True

user = User("Srinadh", 24)

print(user)
print(user.name)
print(user.active)
```

Why this matters in real projects: Dataclasses are useful for DTO-like structures, configuration objects, request/response models, and internal data containers.

Part 6 — Putting OOP Together in a Realistic Example

35. Mini Project: Simple Order System

The following example combines classes, objects, encapsulation, composition, properties, class methods, inheritance, overriding, polymorphism, and validation. It is intentionally small enough to understand but close enough to real application structure to be useful.

```

from abc import ABC, abstractmethod
from dataclasses import dataclass

@dataclass
class Product:
    name: str
    price: float

class PaymentMethod(ABC):
    @abstractmethod
    def pay(self, amount):
        pass

class CardPayment(PaymentMethod):
    def pay(self, amount):
        print(f"Paid ₹{amount:.2f} using card")

class UpiPayment(PaymentMethod):
    def pay(self, amount):
        print(f"Paid ₹{amount:.2f} using UPI")

class Order:
    tax_rate = 0.18

    def __init__(self, customer_name):
        self.customer_name = customer_name
        self._items = []

    @property
    def items(self):
        return list(self._items)

    def add_product(self, product, quantity=1):
        if quantity <= 0:
            raise ValueError("Quantity must be positive")
        self._items.append((product, quantity))

    def subtotal(self):
        return sum(product.price * quantity
                    for product, quantity in self._items)

    def tax(self):
        return self.subtotal() * self.tax_rate

    def total(self):
        return self.subtotal() + self.tax()

    def checkout(self, payment_method):
        amount = self.total()
        payment_method.pay(amount)

```

```

order = Order("Srinadh")

order.add_product(Product("Keyboard", 1500), 1)
order.add_product(Product("Mouse", 800), 2)

print("Subtotal:", order.subtotal())
print("Tax:", order.tax())
print("Total:", order.total())

order.checkout(UpiPayment())
order.checkout(CardPayment())

```

What this example demonstrates:

- Product is a data-oriented class using @dataclass.
- Order owns its list of line items: composition-like containment.
- PaymentMethod defines an abstract interface.
- CardPayment and UpiPayment implement the same interface.
- checkout() uses polymorphism: it only needs an object with pay().
- _items demonstrates controlled internal state.
- @property exposes items without directly exposing the mutable internal list.
- tax_rate is a class attribute shared by orders.

Part 7 — Common OOP Mistakes to Avoid

Making everything a class: Not every function or data structure needs a class. Use a class when state + behavior or a meaningful domain object justifies it.

Using inheritance just for code reuse: If the relationship is not genuinely 'is-a', composition is often cleaner.

Huge classes: A class that handles authentication, payments, emails, database access, and reporting probably has too many responsibilities.

Publicly changing critical state: Use methods/properties to preserve rules such as balance ≥ 0 or status transitions.

Overusing static methods: If a method needs object state, it should probably be an instance method.

Deep inheritance trees: They make behavior difficult to trace. Prefer shallow hierarchies and composition.

Ignoring exceptions: External systems fail. Validate inputs and handle expected errors.

Using mutable class attributes accidentally: A class-level list/dict is shared across instances unless intentionally designed that way.

Confusing class and object: The class is the definition; the object is a particular instance created from it.

Part 8 — How to Read OOP Code in a Real Project

When you open an unfamiliar Python project, use this sequence:

1. Find the class definitions.

- For every class, identify its constructor (`__init__`) and important attributes.
- List the public methods and ask what responsibility each method has.
- Look for inheritance: `class Child(Parent)`.
- Look for composition: `self.some_object = OtherClass(...)`.
- Look for decorators such as `@property`, `@classmethod`, `@staticmethod`, and `@abstractmethod`.
- Look for special methods such as `__str__`, `__repr__`, `__eq__`, `__len__`, and `__iter__`.
- Trace where objects are created and passed into other functions/classes.
- Identify interfaces: which different classes expose the same method?
- Ask: what state does this object own, and what behavior is responsible for changing that state?

Part 9 — Quick Theory for Interviews

Question	Short Answer
What is a class?	A blueprint/definition for creating objects, containing data definitions and behavior.
What is an object?	An instance of a class with its own state.
What is self?	A reference to the current instance passed to normal instance methods.
What is <code>__init__</code> ?	The initializer called after an object is created to establish its initial state.
What is encapsulation?	Bundling state and behavior together while controlling how state is accessed or changed.
What is abstraction?	Exposing essential behavior while hiding implementation details.
What is inheritance?	A mechanism for deriving a child class from a parent class and reusing/extending behavior.
What is polymorphism?	Using a common interface/operation with different object implementations.
What is method overriding?	A child class provides its own implementation of a parent method.
Does Python support method overloading?	Not traditional signature-based overloading; Python commonly uses default arguments, <code>*args/**kwargs</code> , or other dispatch techniques.
Class method vs static method?	A class method receives <code>cls</code> and can work with class state; a static method receives neither <code>self</code> nor <code>cls</code> automatically.
Composition vs inheritance?	Composition models 'has-a' relationships; inheritance models 'is-a' relationships.
What is duck typing?	Python focuses on whether an object supports the required behavior rather than requiring a specific declared type.

Part 10 — Practical Exercises

Do these without looking at the examples. The goal is retrieval practice.

- Create a `Student` class with `name`, `marks`, and an `is_passed()` method.
- Create a `BankAccount` class with `deposit()`, `withdraw()`, and a `balance` property. Prevent negative withdrawals and invalid amounts.
- Create an `Employee` class and `Developer/Designer` subclasses. Override a `describe()` method.
- Create a `PaymentMethod` abstract base class with `CardPayment` and `UpiPayment` implementations.
- Create a `ShoppingCart` class that contains `Product` objects and calculates `subtotal`, `tax`, and `total`.

16. Create a Logger class with a class attribute controlling the application name and a class method for creating a configured logger.
17. Create a custom Playlist class implementing `__len__` and `__iter__`.
18. Refactor one procedural program you already wrote into classes. Identify what state belongs to each object and what behavior belongs with it.

Final Mental Model

If you remember only one framework, remember this:

- Class = blueprint
- Object = actual instance
- Attributes = object/class data
- Methods = behavior
- `self` = current object
- Encapsulation = control and protect state
- Abstraction = expose what matters, hide how it works
- Inheritance = reuse/extend an 'is-a' relationship
- Polymorphism = same interface, different behavior
- Composition = build bigger objects from smaller objects
- Properties = attribute-like access with logic
- Dunder methods = integrate your objects with Python's syntax

The goal is not to memorize OOP terminology. The goal is to look at a problem and naturally decide: what are the objects, what state do they own, what behavior belongs to them, and how should they collaborate?